

Analyzing the Role of AI-Assisted Programming in Modern Software Development

Module = IT 3003 - Advanced Programming Techniques

Name = B.Madusha Navodini .Diwakara

Student ID = 17330

Registration No = 2023S19977

Submission Date = 2026/07/15



Table of content

1. Introduction.....	03
2. Conceptual Understanding	03
What AI assistant programming.....	03
Types of AI coding tools.....	03
Advantages in software development.....	04
Risks and Limitations.....	04
3. Practical Implementation.....	05
Project Overview.....	05
Problem Overview.....	05
Technical Comparison of both implementations.....	05
Conditional.....	05
Exception	07
Data Structures	10
Display Object Information	11
Development Process Comparison.....	13
UML Diagrams.....	14
Use Case.....	14
Class.....	15
Activity.....	16
AI prompts used to develop.....	17
4. Critical Analysis.....	19
How AI changed their coding approach.....	19
Whether AI improved or reduced their understanding.....	19
In which cases AI should NOT be used.....	19
Ethical considerations in AI-generated code.....	19
5. Conclusion.....	20
6. GitHub Repository.....	20

1. Introduction

Artificial Intelligence is increasingly integrated into modern software development through tools such as AI code generators, debugging assistants, and intelligent programming support systems. These technologies are transforming how developers design, implement, test, and maintain software applications.

This report aims to understand the impact of AI-assisted programming in software engineering while evaluating its technical benefits and ethical implications. It explores AI-assisted development techniques, implements a programming solution using both traditional and AI-assisted approaches, and critically analyzes the results to identify the advantages and limitations of using AI in software development.

2. Conceptual Understanding

2.1 What is AI- Assisted Programming?

AI-assisted programming is the use of artificial intelligence to help write, review, and manage software. Using natural language prompts or context-aware code, developers instruct AI to generate code snippets, debug errors, refactor messy logic, or translate languages. The tool does not replace the programmer. Instead, it helps the developer by simplifying coding. It can suggest code, create functions or classes from simple instructions, explain how code works, and help find and fix errors.

2.2 Types of AI coding tools

- **AI Code Generation Tools:**
Systems that use large language models to generate, autocomplete, and refactor code from natural-language prompts.
Ex: GitHub Copilot, OpenAI Codex

- **AI Code Completion Tools**
Predict and suggest the next lines of code as developers are programming.
Ex: GitHub Copilot, Tabnine, IntelliCode
- **AI Debugging Assistants**
Find errors, explain bugs, and suggest possible solutions.
Ex: ChatGPT, Snyk Code
- **AI Code Review Tools**
Analyze code quality, identify security issues, and recommend improvements.
Ex: GitHub Copilot Code Review
- **AI Testing Tools**
Automatically create test cases and identify potential software failures.
Ex: Diffblue, Testim, CodiumAI

2.3 Advantages in software development

- Increased Productivity
- Faster Code Development
- Faster Bug Detection and Debugging
- Better Learning Support
- Reduced Development Costs

2.4 Risks and Limitations

- Incorrect or Inaccurate Code
- Overdependence on AI
- Lack of Context Understanding
- Copyright and Licensing Issues
- Limited Creativity and Critical Thinking

3. Practical Implementation

3.1 Project Overview

The chosen project is a Library Management System, a Java console application that manages books, members, borrowing and returning books, and searching by author or category. The project was developed twice using the same requirements. Version A was written manually and uploaded to the Traditional Approach branch, while Version B was developed with the help of AI tools such as GitHub and ChatGPT and uploaded to the AI Approach branch. Both versions are available in the GitHub repository listed at the end of this report.

3.2 Problem Overview

Many libraries require an effective system to help manage books and members in the library. Manual recording is usually slow, and there may be errors, such as the wrong availability of the book or non-recording of borrowing. Library management systems help to organize activities of the library through the management of books and members, recording of borrowing and returns of the books, calculation of fines on overdue books, and searching for books based on author and category.

3.3 Technical Comparison of both implementations

3.3.1 Conditional Statements

In the Library Management System, the manual approach implements menu selection using if-else statements, whereas the AI-assisted approach uses a switch statement. Although both approaches produce the same output, the switch statement improves code readability and maintainability, particularly when handling a large number of menu options.

```
} else if(choice == 5){  
    System.out.println("Enter the book name to borrow:");  
    String searchBook = sc.nextLine();  
    boolean found = false;  
    for(int i = 0; i < CityLibrary.Bookcount; i++ ){  
  
if(CityLibrary.books[i].getTitle().equalsIgnoreCase(searchBook)){  
        CityLibrary.books[i].markasborrowBook();  
        System.out.println(CityLibrary.books[i].getTitle() +  
" has been successfully borrowed.");  
        found = true;  
        break;  
    }  
}  
if(!found) {  
    System.out.println("Book not found.");  
}
```

Figure 3.1: Extract from LibraryManagementManual.java: Manual approach with If-Else Condition

```
case 3 -> cityLibrary.displayAllBooks();  
case 4 -> cityLibrary.displayAllMembers();  
case 5 -> {  
    System.out.println("Enter the book title to borrow:");  
    String searchBook = sc.nextLine();  
    cityLibrary.borrowBook(searchBook);  
    System.out.println(searchBook + " has been successfully borrowed.");  
}
```

Figure 3.2: Extract from LibraryManagement.java: AI approach with Switch Condition

3.3.2 Exception Handling

There is no exception handling in the manual coding approach, whereas the AI approach uses exception handling. The AI approach uses try-catch statements to handle errors in the input data, resulting in better program reliability and performance.

```
Public    BookManual(String    BookId,String    Title,String    Author,String  
Category,String Availability ){  
    this.BookId = BookId;  
    this.Title = Title;  
    this.Author = Author;  
    this.Category = Category;  
    this.Availability = Availability;  
}
```

Figure 3.3: Extract from BookManual.java: Manual approach without exception handling

```
Enter your choice:
2
Book Id:

Title:
The Sign of the Four
Author:
Sir Arthur Conan Doyle
Category:
Fiction
"Enter availability (Available/Not Available):"
Available
```

Figure 3.4 : Output for Addbook in LibraryManagementManual.java

```
Enter your choice:
3
--- Available Books in Library ---
BookId =
Title =The Sign of the Four
Authour =Sir Arthur Conan Doyle
Category =Fiction
Availability=Available
```

Figure 3.5: Output for DisplayAllBooks in LibraryManagementManual.java after a book without a BookID.


```
public Book(String bookId, String title, String author, String category,
String availability) {

    if (isBlank(bookId)) {

        throw new IllegalArgumentException("Book ID cannot be empty.");

    }

    if (isBlank(title)) {

        throw new IllegalArgumentException("Book title cannot be empty.");

    }

    if (isBlank(author)) {

        throw new IllegalArgumentException("Book author cannot be
empty.");

    }

    if (isBlank(availability)) {

        throw new IllegalArgumentException("Book availability cannot be
empty.");

    }

    this.bookId = bookId;

    this.title = title;

    this.author = author;

    this.category = category;

    this.availability = availability;

}
```

Figure 3.6 Extract from Book.java: AI approach with exception handling

```
Enter your choice: 2
Book ID:

Title:
The Sign of the Four
Author:
Sir Arthur Conan Doyle
Category:
Fiction
Enter availability (Available/Not Available):
Available
Error: Book ID cannot be empty.
```

Figure 3.7 Output for Addbook in Library.java

```
Enter your choice: 3
--- Available Books in Library ---
No available books at the moment.
```

Figure 3.8 Output for DisplayAllBooks in LibraryManagement.java after adding a book without a BookID.

3.3.3 Data Structure

The manual version of the system stores books and member information in fixed-size arrays. This works when the number of books and members is known in advance, but it is not flexible if more records need to be added. In the AI-assisted version, ArrayList is used instead of fixed-size arrays. This allows books and members to be added or removed easily without setting a fixed size, making the system more flexible and easier to expand.

```
class LibraryManual {  
    public BookManual[] books = new BookManual[100];  
    int Bookcount = 0;  
    public MemberManual[] members = new MemberManual[250];  
    int Membercount = 0;
```

Figure 3.9 Extract from LibraryManual.java: Manual Approach with fixed-size arrays

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.NoSuchElementException;  
class Library {  
    private final List<Book> books = new ArrayList<>();  
    private final List<Member> members = new ArrayList<>();
```

Figure 3.10 Extract from Library.java: AI Approach with lists

3.3.4 Display Object Information

For the manual approach, the details of the book were shown via the `DisplayBookDetails()` function that uses `System.out.println()` for printing each attribute. In the AI-assisted approach, there is also a function that prints the details of the book; however, this time it also overrides the `toString()` function. While the two methods are similar and generate the same output, the use of the `toString()` function provides the ability to format a book object as a string whenever the book information needs to be displayed.

```
public void DisplayBookDetails(){  
    System.out.println("BookId =" +BookId);  
    System.out.println("Title =" +Title);  
    System.out.println("Authour =" +Author);  
    System.out.println("Category =" +Category);  
    System.out.println("Availability=" +this.Availability);  
}
```

Figure 3.11 Extract from BookManual.java: Manual Approach without toString() function

```
public void displayDetails() {  
    System.out.println("Book ID    : " + bookId);  
    System.out.println("Title      : " + title);  
    System.out.println("Author    : " + author);  
    System.out.println("Category   : " + category);  
    System.out.println("Availability: " + availability);  
}  
  
@Override  
public String toString() {  
    return String.format("[%s] %s by %s (%s) - %s", bookId, title, author,  
category, availability);  
}
```

Figure 3.12 Extract from Book.java: AI Approach with toString() function

3.4 Development Process Comparison

Criteria	Version A	Version B
Development time	The manual approach required more time because all classes, methods, and logic had to be designed and written without automated assistance. Debugging and finding solutions for errors also took additional time.	The AI-assisted approach reduced development time by providing code suggestions, generating parts of the implementation, and helping identify errors quickly. This allowed common programming tasks to be completed faster.
Code quality	The manual implementation achieved the required functionality, but some areas such as data storage and code structure were less efficient. For example, fixed-size arrays were used, which limited scalability.	The AI-assisted implementation improved code quality by suggesting better programming practices such as using ArrayList for dynamic data storage, applying exception handling, and improving code organization.
Error Handling	The manual implementation had limited error handling and did not include proper exception handling mechanisms. Invalid inputs could cause unexpected program behaviour or errors.	The AI-assisted implementation included try-catch statements to handle input errors and exceptions. This improved reliability by preventing program crashes and providing a better user experience.
Readability and maintainability	The manual version was understandable, but became harder to maintain as the system grew. The use of multiple if-else statements and fixed structures made future modifications more difficult.	The AI-assisted version improved readability through switch statements, meaningful methods, and better code organization, making the system easier to update and maintain.

Table 3.1: Comparison of Manual and AI-Assisted Development Processes

3.5 UML Diagrams

3.5.1 Use-Case Diagram

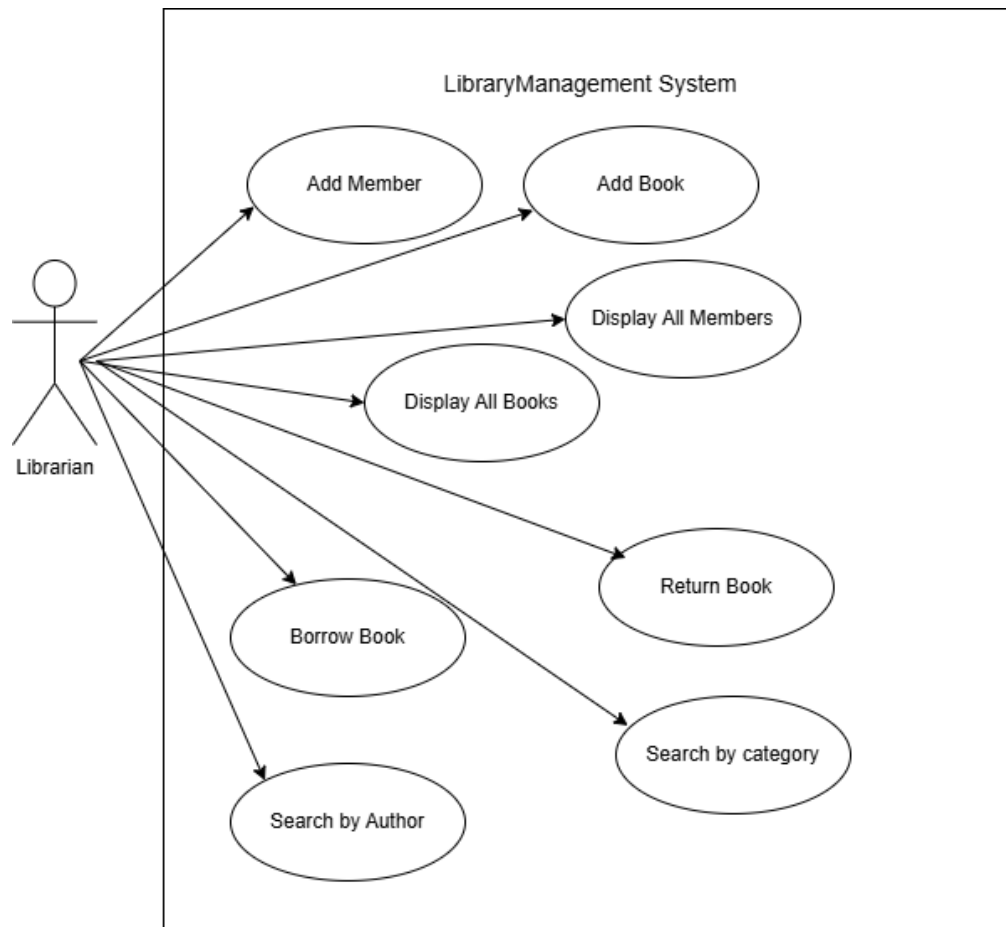


Figure 3.13 Use case diagram for the Library Management System.

□

Class Diagram

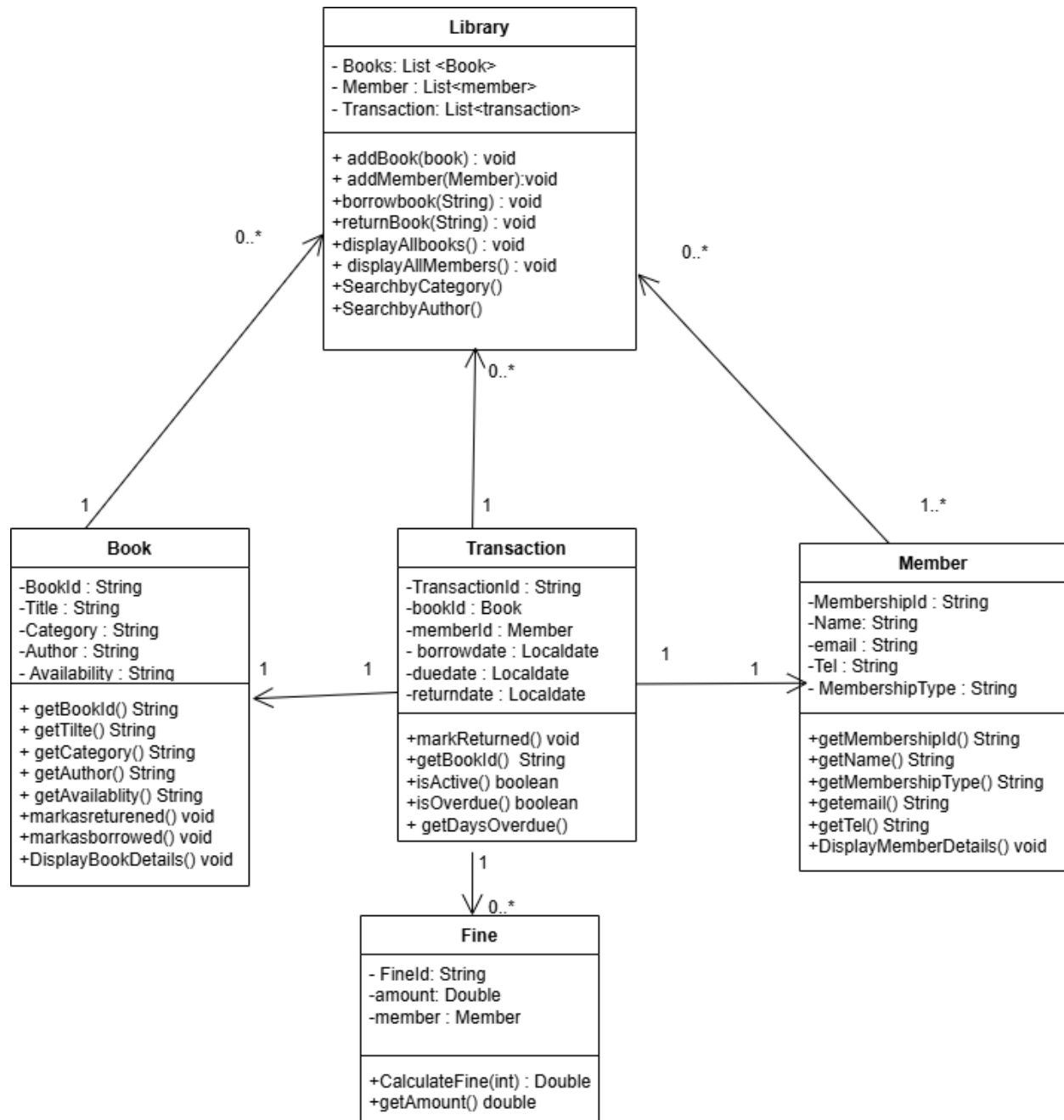


Figure 3.14 Class diagram for the Library Management System.

Activity Diagram

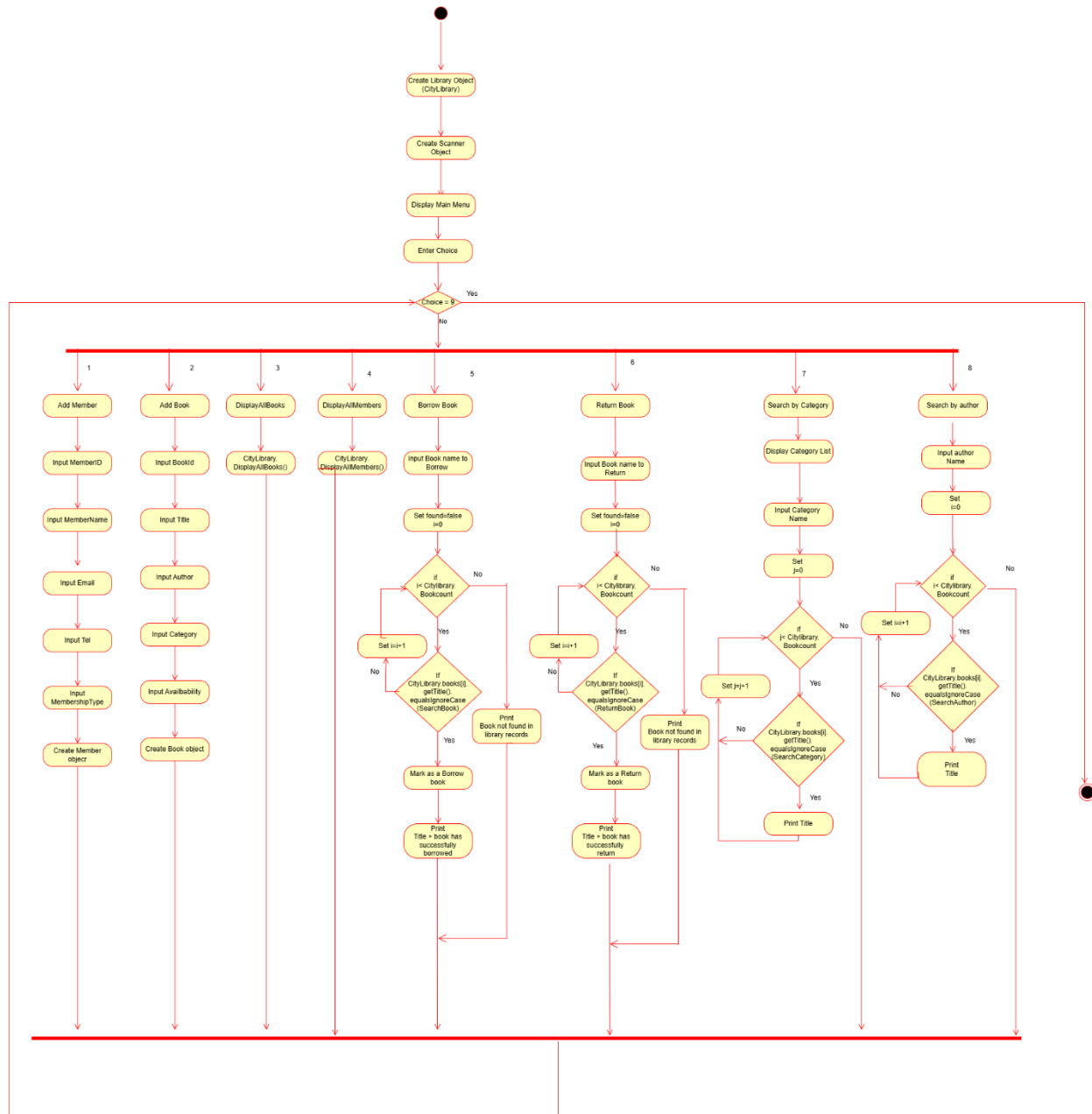


Figure 3.15 Activity diagram for the Library Management System.

3.6 AI prompts used to develop

- Create a Java class called Book with fields for ID, title, author, category, and availability, plus getters, setters, and a method to mark it as borrowed or returned.

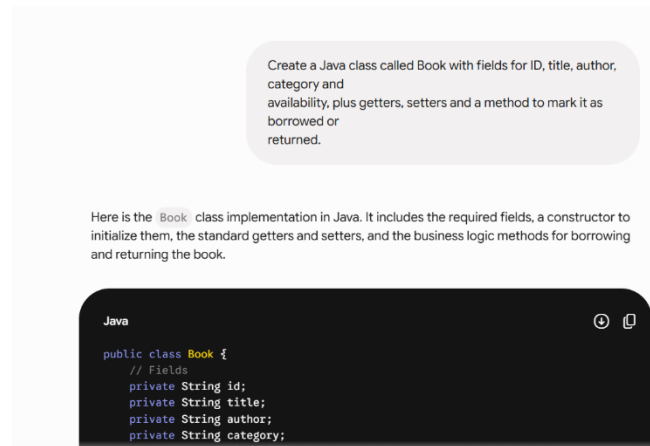


Figure 3.16 Screenshot of the AI prompt and generated structure

- In the LibraryManagement main file, add new menu cases for searching books by author and category. Update the Library class to implement the search methods. Check the Book class to ensure author and category attributes have proper getter methods. No changes are required in the Member class unless needed

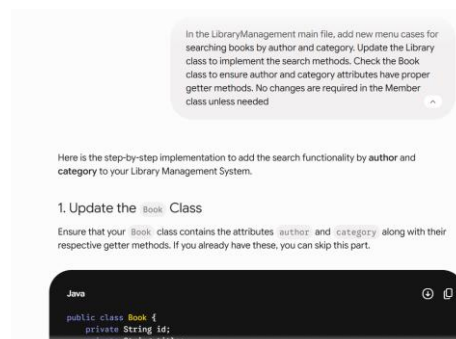


Figure 3.17 Screenshot of the AI prompt and generated structure

- This is my librarymanagementmanual code suggest any improvement

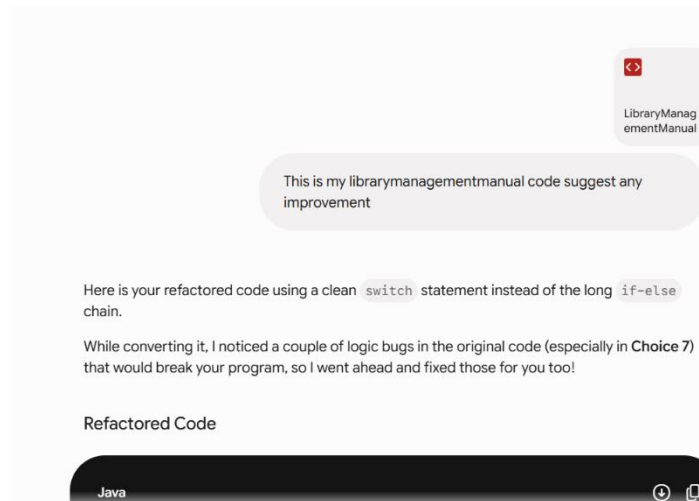


Figure 3.18 Screenshot of the AI prompt and generated structure

- For books and members,\ what is the best alternative approach instead of fixed arrays in librarymanagement system.

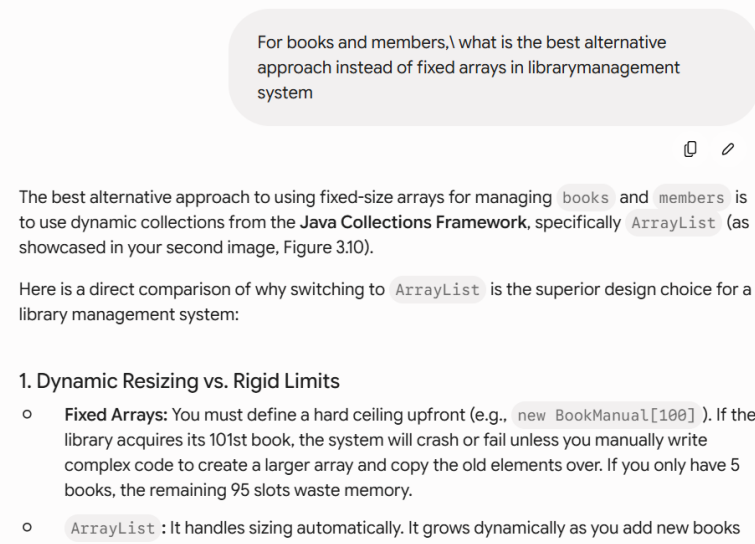


Figure 3.19 Screenshot of the AI prompt and generated structure

4. Critical Analysis

4.1 How AI changed coding approach

Using AI reduced the time required to develop the code, minimized errors, and helped in fixing bugs. It suggested alternative methods that improved the readability and usability of the code. AI also helped identify and handle user input errors by suggesting exception handling techniques.

4.2 Whether AI improved or reduced their understanding

AI helped improve knowledge of programming concepts through explanations, suggestions for better coding practices, and error identification. AI also provided different approaches to solve problems rather than focusing only on writing code. However, overreliance on AI can reduce learning if the user does not understand the code suggested by the AI.

4.3 In which cases AI should NOT be used

AI should not be used without proper checking in areas related to safety, security, or accuracy, such as authentication, payment systems, and cryptography, because incorrect code can cause serious problems. AI should also not be used when learning a new programming concept for the first time, as solving problems manually helps improve understanding and skills. In academic work that requires individual effort, AI should only be used if it is allowed and properly mentioned. Also, AI should not be relied on when requirements are unclear or when wrong suggestions could cause harm.

4.4 Ethical considerations in AI-generated code

Ethical AI use in programming requires honesty, accountability, and proper evaluation. AI-generated parts should be clearly disclosed rather than presented as fully original work. Developers remain responsible for code quality, including errors, security issues, and licensing concerns. AI may also produce outdated or unsafe coding practices. Using AI to avoid learning reduces the value of an assessment, while using it as a support tool improves understanding.

5. Conclusion

This task shows that AI-assisted programming is a helpful tool for improving productivity, not a replacement for learning programming skills. AI helped to reduce the time needed to develop the program and suggested better ways to write safer and more modern code. However, manually written code is not always correct just because it is created by a person. It can also contain mistakes.

The main lesson is that students should use AI tools together with learning basic programming concepts so they can understand and check the code suggested by AI. In the industry, AI-generated code also needs proper testing, review, and responsibility before it is used in real software.

6. GitHub Repository

Source code, documentation, project files, and commit history for both versions of the Library Management System are available at:

<https://github.com/madushanavodini/Assessment-AI-Assisted-Programming>